COMP90041
Programming and Software Development

Lecture 11 - File Handling & Advanced Topics

Danny Xu

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

THE UNIVERSITY OF MELBOURNE

Overview

In this part of the lecture, you will learn about:

- File Handling : Binary Files
- Advanced Topics
  - Try with resources
  - Recursion
  - Lambda Statements
  - Multi-threading
  - GUI

File Handling: Binary Files

Recap

- We looked at handling text files in Week 9
- Files can be read or written into using streams – FileInputStream, FileOutputStream
- For reading
  - Scanner – read from system input or files
  - BufferedReader – more efficient than Scanner, uses a buffer
- For writing
  - PrintWriter – can write to files using print, println, printf methods
  - BufferedWriter – more efficient than PrintWriter, uses a buffer.
- Do not forget to flush the contents after writing.
- Use File class for advanced file/folder operations – like creating folders, manage read/write permissions.

Binary Files

- There are many kinds of binary files
  - Images
  - Audios
  - Videos
  - Compressed files
  - Database files
  - Document files like docx
  - Text files can also be saved as binary files in Java.
- These files can be read from or written into by Java file readers and writers
- But the data is not in a readable format. You need special drivers to look into the content and convert it into a meaninful format.

Binary Files in Java

- Binary Files in Java only contain information in bits. A bit is 0 or 1.
- 8 bits = 1 byte.
- Your computer memory comes with 32 bits or 64 bits. An operating system decides how to load the data. There are two formats – Big Endian & Little Endian
- For example – you want to load 32-bit (4 bytes) data – 12 ae 5b f6
  - Big Endian - | 12 | ae | 5b | f6 | ⟵ starts from the left-hand side
  - Little Endian - | f6 | 5b | ae | 12 | ⟵ starts from the right-hand side
- Java runtime can handle the binary files in both formats.
  - A binary file created by a Java program can be moved from one computer to another.
  - These files can then be read by a Java program with the same object definitions.

Writing Binary Files in Java

- Use ObjectOutputStream instead of `PrintWriter`.
- Similar to `PrintWriter` takes a `new FileOutputStream(fileName);`
- Use the `writeInt, writeDouble, writeUTF, writeObject`, not `print, printf` or `println` methods.
- Remember to flush and close the streams.
- Exceptions :
  - `FileOutputStream` throws a `FileNotFoundException` if file is not ready for writing.

```
ObjectOutputStream printer= null, FileOutputStream fos = null;
try{
    fos = new FileOutputStream("C:\\output.txt");
    printer= new ObjectOutputStream(fos);
    printer.writeChars("This is a text");
    printer.writeInt(23);
 }catch(Exception ex){
    // handle Exceptions
 }finally{
    if (printer != null) // good practice
      printer.close(); // don't forget to close the fos as well in finally
}
```

Reading Binary Files in Java

- Use ObjectInputStream instead of `Scanner`.
- Similar to `Scanner` takes a `new FileInputStream(fileName);`
- Use the `readInt, readDouble, readUTF, readObject` methods.
- Exceptions :
  - `FileInputStream` throws a `FileNotFoundException` if file is not ready for writing.
  - `ObjectInputStream` throws an `IOException` if the stream is not valid.

```
ObjectInputStream reader= null, FileInputStream fis = null;
try{
    fis = new FileInputStream("C:\\input.txt");
    reader= new ObjectInputStream(fis);
    while(reader.available() > 0) // checks bytes are available to read
        System.out.println(reader.readLine());
 }catch(Exception ex){
    // handle Exceptions
 }finally{
    if (reader!= null) // good practice
      reader.close(); // don't forget to close the fos as well in finally
}
```

Other methods for reading/writing

- You can read some characters/bytes using `skipBytes() method`
- Some bytes may not translate correctly into characters. There is a particular range of bytes that are represented as a valid character. A character set could be ISO-8089-1, UTF-8 (most used) or UTF-16 (contains Chinese characters and other languages as well). You can use `readUTF/writeUTF methods`
- You can directly read or write an object into a file. Think of saving employee records in a binary file. Your own personal mini database without the database features though. Use the `readObject/writeObject methods`
- Full list of documentation present here -
  - OutputStream – Javadocs here.
  - InputStream – Javadocs here.

Random Access File (Advanced/Non-examinable)

- Until now, we open a file and start reading from the first byte/position.
- What if we need to read/write somewhere from the middle.
- RandomAccessFile in java supports **random read/write access** to files (unlike sequential streams).
- Uses a **file pointer** to mark the current read/write position; pointer can be moved with `seek()`.
- Allows reading/writing of **primitive data types** and strings using methods like `readInt()`, `writeDouble()`, `readUTF()`, etc.
- File is opened with modes like `"r"` (read-only) or `"rw"` (read/write); existing data is preserved.
- File size can be modified using `setLength()`, enabling truncation or extension.
- Commonly used in **large database applications** requiring fast direct access to file records.
- It does **not support object serialization** (`readObject` / `writeObject`), unlike ObjectInputStream/ObjectOutputStream.
- File updates using `"rws"` or `"rwd"` modes ensure changes are **synchronously written** to storage (advanced use cases).

Advanced Topics (Non-examinable)

Try-with-resources

- Closing IO streams is important in Java.
  - File readers/writers and streams in Java implements the Closeable interface.
  - close() method is used to close these resources in the finally block.
  - And sometimes you forget to close the resources!
- New Java versions has AutoCloseable interface
  - Allows try-with-resources blocks to automatically close the streams without explicit close() method call.
  - No finally block needed unless you want to do something else like setting default values.

```
try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
  String line;
  while ((line = reader.readLine()) != null) {
    System.out.println(line);
  }
} catch (IOException e) {
  e.printStackTrace();
}
```

Recursion

- A method calling **itself** to solve a smaller instance of the same problem.
- Two of the easiest examples to visualise recursion as concepts are -
  - Factorials
  - Fibonacci

- Factorials – n! = n x (n-1)! Each time we reduce the value by 1 and find factorial of n-1.
- Fibonacci series – Fibonacci number starts from 0 and 1. Next number is sum of last two numbers.
  - 0, 1, 1, 2, 3, 5, 8, 13.....
  - How to calculate the 6th Fibonacci number, i.e,. 5 in the series
    - Find 5th and 4th number and add up
      - How to find 5th number?
        - Find 4th and 3rd number and add up
          - How to find 4th number
            - Find 3rd and 2nd number and add up.
              - How to find 3rd number
                - Find 2nd and 1st number and add up i.e 0+ 1 = 1
          - 4th number is 1 + 1 = 2
      - 5th number 2 +1 = 3
  - 6th number = 3 + 3 = 5

Recursion

- If the method calls itself, then when does it stop?
- If it doesn't stop then it will go in the infinite loop not yielding any output.
- How to write recursion – the structure of recursion method
  - Recursion method should return something.
  - Use if condition to meet the stopping criteria – Stops the recursion returns a value
  - Recursive Case – Method continues calling itself with smaller inputs.

```
public static int factorial(int number) {
   if (number == 0) {  // Base case
      return 1;
   } else {
      return number * factorial(number - 1);// Recursive call with modified input
   }
}

public static int fibonacci(int number) {
   if (number <= 1) {  // Base case
      return number;
   } else {
      return fibonacci(number - 1) + fibonacci(number - 2); // Recursive call
   }
}
```

Lambda Statements

- Functional programming treats computations as data transformations using functions.
- Java 8 introduced lambda expressions to support functional programming.
- Enables writing cleaner, concise, and anonymous functions.
- Java supports functional-style programming using functional interfaces and lambda expressions.
- Lambdas are nameless (anonymous) function used to implement functional interfaces.
  - Simplifies code for event handling, filtering, iteration, etc.
  - Improves readability for short operations.
  - Powers features like Streams API, functional interfaces, and method references.
- Example – Collections have a sort method but needs Comparator Interface method. Simple comparisons like Integer or String comparisons can be written in just one line
  - `Collections.sort(nameList, (String s1, String s2) -> s1.compareTo(s2));`
  - `Collections.sort(studentIdList, (Integer studentId1, Integer studentId2) -> Integer.compare(studentId1, studentId2));`

GUI

- Java has many advanced UI libraries. There are two core ones -
- AWT (Abstract Window Toolkit)
  - Foundation of Java GUI development; part of Java since the beginning.
  - Uses native OS components, making it platform-dependent in look and behavior.
  - Provides basic GUI elements: buttons, labels, text fields, etc.
  - Simpler but more limited in functionality and flexibility.
  - Suitable for lightweight applications or understanding GUI fundamentals
- Swing
  - Advanced GUI toolkit built on top of AWT (introduced in Java 1.2).
  - Offers a richer set of components: tables, trees, tabbed panes, styled text, etc.
  - Components are lightweight (not tied to native OS UI), resulting in consistent cross-platform look.
  - Includes flexible layout managers for responsive designs.
  - Supports event-driven programming, custom UI rendering, and MVC patterns.
  - Good choice for modern Java desktop applications with advanced features from Java 17 onwards.

Multi-threading

- Multithreading is the ability of a CPU (or a single core) to execute multiple threads concurrently.
- Each thread represents an independent path of execution.
- Java supports multithreading via:
  - `Thread` class
  - `Runnable` interface
- Why use Multithreading?
  - Improves application responsiveness.
  - Allows for parallel execution of tasks.
  - Efficient use of CPU cores.
  - Ideal for background tasks (e.g., I/O, animations, file processing).

Multi-threading

- Thread Lifecycle
  - New – Thread is created.
  - Runnable – Thread is ready to run.
  - Running – Thread is executing.
  - Blocked/Waiting – Waiting for a resource or signal.
  - Terminated – Execution is complete.

- Multiple threads accessing shared data → can lead to **race conditions**.
- Use `synchronized` keyword to prevent **data inconsistency**:

Live Coding

Live Coding

- Combined example of Inheritance/Interfaces/Generics
  - Garage Demo

- Multi-threading with Collections
  - Add tasks to todo list
  - Delete when done
  - Show them running concurrently in thread.

Additional Reading Resources

- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 10. 10.4)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 22 under Serialization)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 15 Lambda Expressions Introduction, Chapter 11 Multi-threading, Chapter 32 GUI)

Copyright Notice

The University of Melbourne (Australian University)
PRV12150/CRICOS 00116K

See you next time!

The University of Melbourne (Australian University) PRV12150/CRICOS 00116K